DNSSEC_CacheFlush_Mitigations

Below is one “creative” technique to publish massively large RRSIG records under a secure (DNSSEC-enabled) domain with a valid DS—while effectively skipping validation for the huge signature itself. The trick relies on publishing multiple DNSKEYs and multiple RRSIGs under different DNSSEC algorithms—at least one of which is recognized (so the zone validates overall) and one (or more) that is unknown or unsupported by the resolver (so that particular RRSIG is never actually validated). The resolver still treats the zone as secure, but it will cache the unvalidated (and potentially huge) RRSIG for the unknown algorithm.

1. Background: Multiple DNSSEC Algorithms in One Zone
DNSSEC allows a zone to be signed simultaneously by multiple DNSKEYs, each potentially using a different cryptographic algorithm (e.g., RSA/SHA-256, ECDSA, Ed25519, etc.). For each DNSKEY, you generate a corresponding RRSIG over each RRset.
RFC 6840, Section 5.2, clarifies that if at least one recognized algorithm/path can be validated successfully, the zone is considered secure—even if other algorithms or signatures are unknown or unsupported by the validator.
Put differently:
If a resolver finds a valid chain of trust via any one known algorithm, it marks the data secure. Other unrecognized or unsupported RRSIGs do not cause the zone to become bogus.

2. High-Level Attack/Technique
Keep the Domain Fully Secure
You have a valid DS in the parent zone.
At least one of your DNSKEYs uses a mainstream algorithm (e.g., RSA/SHA-256) that the resolver recognizes.
That legitimate key signs your zone normally, producing standard-sized RRSIGs that fully validate.
Add an Unknown / Unsupported Algorithm
You register a custom or “private-use” DNSSEC algorithm

code (e.g., something the validator doesn't implement).
For example, choose an algorithm number in the "Reserved for Private Use" range, or a newly allocated code that few resolvers support yet.
Publish a second DNSKEY of that unknown algorithm with an absurdly large key size (e.g., 16k–64k bits if RSA-based).
Generate a Gargantuan RRSIG
Using that huge unknown-algo DNSKEY, generate an enormous signature (the RRSIG).
Because RSA signature size scales roughly with the key size, you can create multi-kilobyte (or near-65k) RRSIG RDATA.
Zone Has Two RRSIGs per RRset
RRSIG #1 (Recognized Algo): e.g., RSASHA256 or ECDSA. The resolver can and will validate this.
RRSIG #2 (Unknown/Unsupported Algo): e.g., "Algorithm 253/254" or some not-yet-implemented algorithm ID. This one has a massive signature field.
Resolver Behavior
Because there is a valid DS in the parent and at least one RRSIG from a recognized algorithm, the zone passes DNSSEC validation overall.
The "unknown algorithm" RRSIG is effectively ignored for validation. The resolver sees it, doesn't know how to validate it, but does not treat the zone as bogus.
Typically, the resolver caches all RRs returned from the authoritative server—even those it did not use in successful validation—unless it has specific code to discard them.
Result: The monstrous RRSIG is stored in the resolver's cache.
Hence, you have a fully secure domain (DS present, recognized key is valid) while shipping a huge unvalidated RRSIG that lands in the resolver's cache.

3. Why This Works
DNSSEC "At-Least-One" Validation Model
Under DNSSEC standards, if any one recognized crypto algorithm's signature is valid, the resolver must treat the zone as secure. An unrecognized signature cannot make an otherwise valid zone become bogus.
Caching of Extra RRsets
By default, most resolvers store all data in a response if it isn't obviously bogus or out-of-bailiwick. That includes extra

RRSIGs for algorithms they don't handle.
Because the zone is successfully validated by the recognized signature, there's no reason for the resolver to discard the "extra" RRSIG. The large RRSIG from the unknown algorithm simply goes unvalidated, but still ends up in the cache.

4. Practical Steps
Obtain/Define a "Private" DNSSEC Algorithm
For testing or malicious demonstration, pick an algorithm number in the "private use" range (e.g., RFC 6014).
Write or modify a signing tool (like ldns-signzone, dnssec-signzone, etc.) to generate a "fake" signature using your large key.
Keep a Normal Algorithm
Use a mainstream algorithm (e.g., RSA/SHA-256, ECDSA P-256, etc.) for the real chain of trust.
Publish that DNSKEY in the parent's DS record so the domain is recognized as "secure."
Maximize the "Unknown" RRSIG
Create an RSA key of extreme size (16k-bit or more).
Generate a signature with it (the "private" algorithm). The signature can be thousands of bytes.
Because an RRSIG's signature field is unbounded (up to the overall DNS 65,535-byte limit), you can approach the maximum packet size.
Push That Huge RRSIG in Your Zone
When you sign each RRset, produce two RRSIGs:
The normal recognized algorithm RRSIG.
The massive unknown algorithm RRSIG.
The authority server returns both in the Answer or Authority section as appropriate.
Resolver Fetches and Caches
During resolution, the resolver sees the DS from the parent, downloads your zone's DNSKEYs, sees one recognized key (and one unknown), checks the recognized signature → success.
The unknown signature is left unvalidated, but not thrown away. It's simply an extra record.
In many implementations (BIND, Unbound, etc.), that extra RRSIG is then stored in the cache for its TTL.

5. Potential Obstacles & Considerations
Implementation Differences
Some resolvers might discard RRSIGs that use unknown algorithms.
Others will keep them around as they do no harm if at least one valid signature was found. Behavior can vary by version and configuration.
Message Size / Fragmentation
A near-65k response will definitely require TCP fallback, which might trigger more stringent checks or rate limits. Some resolvers or middleboxes may drop or truncate extremely large DNS responses.
Possible “Bogus” If the Resolver Does Know the Algorithm
If the resolver unexpectedly supports your “unknown” algorithm but cannot properly validate the signature (e.g., key too large?), it might mark the zone “bogus.” This depends on how strictly the code tries to do the cryptography.
Typically, though, truly “unknown” or “private” algorithm numbers are simply ignored.
Parent DS
Make sure the DS in the parent references only your recognized DNSKEY (the standard algorithm). Do not attempt to reference the unknown key.

Final Summary
Goal: Force a validating resolver (with a real DS chain of trust) to cache a huge RRSIG unvalidated.
Method:
Sign the zone with two algorithms: one standard (to ensure actual validation) and one private/unrecognized (carrying a gigantic signature).
Publish both RRSIGs in each answer.
The resolver validates the known algorithm’s RRSIG, treats the zone as secure, does not discard the unknown RRSIG, and caches the huge signature anyway.
In effect, you have a secure, DS-enabled domain that skips validation on the giant RRSIG while still caching it—achieving the “large, unvalidated data in a secure zone” outcome.

Short Answer:
No. A validating resolver does not need to support (i.e., understand or validate) the unknown algorithm in order to cache that RRSIG. As long as one recognized algorithm validates the zone, the resolver marks the zone as “secure” and retains all the associated RRSIGs (including the unknown-algorithm one) in its cache—even though it never actually validates that unknown signature.

Why the Resolver Caches an “Unknown-Algorithm” RRSIG
DNSSEC’s “At-Least-One” Validation Rule
Under DNSSEC (per RFC 6840, Section 5.2), if the resolver can successfully validate an RRset with any one recognized algorithm, the data is considered secure.
Any additional RRSIGs using algorithms the resolver does not recognize do not cause the zone to become bogus. They are simply ignored for validation purposes.
Resolvers Typically Cache All RR Data in a Validated Zone
Once the resolver determines the zone is secure (because it validated at least one known signature), it normally caches all the records in the response (subject to any local policy or size limit).
This includes any extra RRSIGs (even if the algorithm is unknown or unsupported).
No Need to Parse or Verify the Unknown Signature
The resolver only needs to verify the RRSIG(s) for the algorithm(s) it does support.
For the unknown algorithm’s RRSIG, the resolver does not even attempt validation. It treats it as additional data, not affecting the overall “secure” status of the zone.
Result
The large, unknown-algorithm RRSIG record is stored in the cache with the rest of the zone data, even though it was never validated.

What If the Resolver Did “Support” the Unknown Algorithm?
If the resolver happens to partially support or recognize that algorithm but cannot successfully validate the signature (for instance, due to an impossibly large key or incomplete cryptographic support), there is a risk it could mark the zone as “bogus.”
To ensure that big RRSIG does not break validation, you

want an algorithm code that the resolver truly does not implement. Then, by the DNSSEC spec, the resolver must ignore that signature rather than fail on it.

Practical Caveats
Some Resolvers May Strip or Reject Unknown RRSIGs
Certain resolvers (or custom patches) might have a policy to discard RRSIGs they cannot validate. This is not standard behavior but can happen in hardened or heavily customized DNS setups.
Message Size & Fragmentation
A giant RRSIG could push the DNS response toward 65k bytes. The resolver may fall back to TCP, or large responses might be dropped by firewalls/middleboxes if they exceed certain limits.
Chain of Trust Must Be Valid for at Least One Key
Remember that your domain must publish a DS record that matches the recognized DNSKEY. That key's RRSIG ensures the zone is accepted as secure.

Summary
A validating resolver does not need to support or validate an unknown DNSSEC algorithm to store its RRSIG in cache. As long as the zone successfully validates under a known, recognized algorithm, the resolver treats the zone as "secure" and retains all the RRSIGs (including the big "unknown-algo" one) in its cache. This is exactly how you can publish very large RRSIGs in a secure domain while bypassing validation for those oversized signatures.

Short Answer:
Yes. If the second signature uses an unsupported or "private-use" DNSSEC algorithm, the resolver will not attempt to validate it. From the resolver's perspective, that signature is simply "unrecognized data," so you could fill it with essentially any bytes. As long as the zone does have at least one valid, recognized signature/algorithm, the domain will still be considered "secure," and the bogus (or random) second signature will simply be cached without validation.

Why "Bogus" or Random Data Can Be Used

DNSSEC's Multi-Algorithm Rule

A zone can publish multiple DNSKEYs, each under a different algorithm.

If one (recognized) algorithm validates successfully, the zone is marked "secure." The resolver ignores any unknown or unsupported algorithms—it does not cause the zone to become "bogus."

Unknown Algorithms Are Ignored

Per the DNSSEC specifications (RFC 6840, Section 5.2), if a resolver doesn't implement or recognize the algorithm number, it does not validate that signature.

"Bogus" in DNSSEC terms usually means "recognized but cryptographically invalid." But if the algorithm is unknown, the resolver doesn't even try to validate it, so it never gets flagged as bogus.

Resolver Caches All RRSIGs from a Secure Zone

Once the zone is deemed "secure" by at least one valid signature, the resolver typically caches all RRSIGs included in the response.

That includes the second "unknown-algo" RRSIG—even if it's just random data.

No Impact on Validation

Because the zone does have a DS at the parent and a valid signature from a recognized algorithm, the chain of trust is intact.

The unknown-algo signature does not matter to the resolver's final "secure" vs. "bogus" decision.

Practical Example

One "Legit" Algorithm

Suppose you have a DNSKEY using RSA/SHA-256 (algorithm 8) or ECDSA (algorithm 13), which is widely recognized.

You configure your parent DS to match this key, so a validating resolver can build a chain of trust and confirm the domain is properly signed.

One "Fake" or Unknown Algorithm

You add a second DNSKEY that uses a "private-use" or not-yet-implemented algorithm number (e.g., 253 or 254).

You generate a signature (RRSIG) for that second key, but in reality, you can stick arbitrary or random data in the signature field—there is no standard validation because the resolver doesn't know how.

Resolver Behavior
Upon lookup, the resolver sees two RRSIGs.
RRSIG #1 (RSA/SHA-256): Validates fine → zone is “secure.”
RRSIG #2 (unknown algorithm): Skipped for validation because the algorithm is not recognized.
The resolver proceeds to cache both RRSIG #1 and #2. The second signature is effectively “unvalidated junk,” yet still stored.

Caveats
Some Resolvers May Discard Unknown RRSIGs
Not all resolvers behave identically. Some might drop (not cache) RRSIGs with unknown algorithms. Others might keep them as normal resource records.
Size/Message Limits
If you put truly massive or random data in the second RRSIG, you risk hitting message-size limits or being truncated by middleboxes.
You may need TCP fallback, which some resolvers or firewalls treat more strictly.
Ensure the Recognized Algorithm Truly Validates
If your “legit” signature fails, the zone is bogus overall. The entire zone (including your second signature) becomes irrelevant to resolvers.

Final Takeaway
Yes, that second signature can be entirely “bogus” or random as long as it uses an unknown or unsupported DNSSEC algorithm.
The resolver will only validate the known algorithm’s signature. If that one is good, the zone remains secure, and the junk signature is simply cached without scrutiny.

Learn the conceptual attack above carefully.

You are an DNS security expert.

What would be the most logical mitigation for this conceptual attack that does not affect the other parts of the DNS system?

Mitigating this conceptual attack while preserving the integrity and functionality of the DNS system requires targeted solutions. Here are logical and practical mitigations:

---

1. Limit Caching of Unsupported/Unknown Algorithm RRSIGs

Resolvers could implement policies to exclude RRSIGs with unknown or unsupported algorithms from the cache. This approach aligns with the principle that only validated data should occupy cache space.

- **Implementation Details**:
  - Modify resolver behavior to only cache RRSIGs from algorithms that have been validated.
  - If an RRSIG uses an unknown algorithm, discard it after processing the response.
- **Advantages**:
  - Minimally disruptive to existing DNS functionality.
  - Still complies with the DNSSEC “at-least-one-valid” model.
- **Potential Drawback**:
  - Could marginally increase resolver processing requirements for filtering responses.

---

2. Enforce Size Limits on RRSIG Records

Resolvers could enforce size constraints on individual RRSIG records, preventing excessively large signatures from being cached or processed.

- **Implementation Details**:
  - Define a maximum size for RRSIG RDATA (e.g., a reasonable threshold like 1 KB or 2 KB).
  - Reject or discard RRSIGs exceeding this limit during validation or caching.
- **Advantages**:
  - Protects against potential denial-of-service risks caused by large RRSIGs.

- Does not require changes to DNSSEC validation logic.

- **Potential Drawback**:
  - Some legitimate cryptographic algorithms (with large signatures) might require careful exception handling.

---

3. Cache RRSIGs Selectively Based on Validation

Resolvers could cache only those RRSIGs that contribute to successful validation of the zone.

- **Implementation Details**:
  - After validation, retain only RRSIGs corresponding to recognized and successfully validated algorithms.
  - Discard RRSIGs from unrecognized algorithms as they do not contribute to trust validation.
- **Advantages**:
  - Reduces unnecessary caching of unvalidated data.
  - Maintains compliance with the DNSSEC trust model.
- **Potential Drawback**:
  - Slight increase in validation logic complexity.

---

4. Rate-Limit or Cap Response Sizes for Zones with Unknown Algorithms

Authoritative servers could be configured to avoid overloading resolvers by rate-limiting zones using unknown DNSSEC algorithms.

- **Implementation Details**:
  - Apply rate limits or size caps to DNS responses containing unknown or private-use algorithms.
  - Encourage use of mainstream algorithms for zones requiring wide compatibility.

- **Advantages**:
  - Reduces potential abuse while allowing for controlled testing of new algorithms.
- **Potential Drawback**:
  - Authoritative server policy changes might not address all resolver behaviors directly.

---

5. Resolver Behavioral Updates for Unknown Algorithms

Resolvers could explicitly handle responses with unknown algorithms more robustly, ensuring unknown RRSIGs do not create unintended consequences.

- **Implementation Details**:
  - When encountering an unknown algorithm, log and ignore the corresponding RRSIG but do not cache it.
  - Notify administrators of suspicious configurations if multiple unknown algorithms are observed.
- **Advantages**:
  - Preserves flexibility for future algorithms while preventing abuse.
  - Adds a level of monitoring and visibility for DNS administrators.
- **Potential Drawback**:
  - Requires updates to resolver software across ecosystems.

---

6. Encourage Cryptographic Algorithm Agility

Promote cryptographic agility by encouraging zones to adopt and support modern, efficient, and widely supported DNSSEC algorithms.

- **Implementation Details**:
  - Deprecate or discourage use of outdated or excessively large algorithms (e.g., large RSA keys).

- Encourage adoption of efficient elliptic-curve cryptography (e.g., Ed25519, ECDSA P-256).

- **Advantages**:
  - Reduces the attack surface for such conceptual exploits.
  - Aligns with overall best practices in cryptographic evolution.
- **Potential Drawback**:
  - Requires long-term adoption efforts and education for DNS administrators.

---

7. Combine Size Limits with Unknown Algorithm Handling

A hybrid approach where unknown algorithm RRSIGs are both limited in size and excluded from caching ensures resilience against this attack while preserving standards compliance.

- **Implementation Details**:
  - Resolvers enforce both size thresholds and validation-based caching rules for RRSIGs.
  - Implement fallback behaviors to ensure valid zones remain resolvable even in edge cases.
- **Advantages**:
  - Comprehensive protection against oversized or malicious RRSIGs.
  - Future-proof against algorithm mismatches or misconfigurations.
- **Potential Drawback**:
  - Slightly more complex implementation in resolver code.

Conclusion

The most logical mitigation would be a **combination of enforcing size limits on RRSIGs and caching only validated signatures**. This dual approach ensures that oversized or malicious RRSIGs are not cached while maintaining compatibility with the DNSSEC “at-least-one-valid”

model. It addresses the core issue without disrupting legitimate DNS operations or requiring widespread system changes.

> Learn the alternative version of the attack given below:
>
> Multiple Signatures: For example, the RSA-4096 algorithm produces 744-byte DNSSEC signatures. With 57 DNSKEYs and signatures, a response can reach approximately 65 KB.
>
> This can be still used to flush the benign cache of the resolvers.
>
> You are an DNS security expert.
>
> What would be the most logical mitigation for this conceptual attack that does not affect the other parts of the DNS system?

To mitigate this alternative version of the attack, which uses multiple valid signatures (e.g., RSA-4096) to fill resolver caches and potentially cause cache exhaustion, the most logical and targeted mitigation strategies would focus on limiting the impact of excessive resource record sets (RRsets) without disrupting legitimate DNS operations.

---

1. Limit the Number of Signatures Per RRset

Resolvers could impose a cap on the number of DNSKEYs or RRSIGs per RRset that are processed or cached.

- **Implementation Details**:
  - Set a reasonable maximum number of DNSKEYs or RRSIGs per RRset (e.g., 10–20).
  - Ignore or discard additional signatures beyond the configured limit during validation or caching.
- **Advantages**:
  - Directly limits the attack vector of excessive signatures.

- Has minimal impact on legitimate zones that do not require an excessive number of keys.

- **Potential Drawback**:
  - Might affect zones with legitimate use cases for large numbers of keys or signatures.

---

2. Enforce a Total Size Limit for DNS Responses

Resolvers could enforce size constraints on DNS responses at various stages to prevent cache exhaustion.

- **Implementation Details**:
  - Impose a maximum size for RRsets or full DNS responses (e.g., 4 KB–8 KB) for resolver caching purposes.
  - Drop excessive responses or truncate them before caching.
- **Advantages**:
  - Prevents oversized responses from overwhelming resolver caches or causing fragmentation issues.
  - Simple to implement without disrupting DNSSEC validation logic.
- **Potential Drawback**:
  - May require fallback mechanisms or revalidation for truncated responses.

---

3. Prioritize Cache Entries Based on Utility

Resolvers could prioritize caching based on the utility and frequency of lookups, discarding less-used or excessive entries.

- **Implementation Details**:
  - Implement a least-recently-used (LRU) or similar cache eviction policy.

    - Prioritize commonly queried domains over zones with excessive or malicious signatures.
- **Advantages**:
    - Ensures critical and frequently used zones remain in cache.
    - Reduces impact of cache exhaustion without requiring strict limits on RRsets.
- **Potential Drawback**:
    - Complex cache management algorithms may increase resolver overhead slightly.

---

4. Penalize Zones with Excessive DNSKEYs or RRSIGs

Resolvers could flag or deprioritize zones with an unusually high number of DNSKEYs or RRSIGs, treating them as potentially abusive.

- **Implementation Details**:
    - Introduce rate-limiting or query penalties for zones with excessive DNSSEC overhead.
    - Consider dynamic thresholds based on observed behavior (e.g., average number of signatures per zone).
- **Advantages**:
    - Discourages abuse without affecting standard zones.
    - Adapts dynamically to evolving DNSSEC use cases.
- **Potential Drawback**:
    - May unfairly penalize legitimate but unusually configured zones.

---

5. Encourage Use of Compact and Efficient Algorithms

DNS administrators should be encouraged to use cryptographic algorithms that produce smaller signatures (e.g., ECDSA or Ed25519).

- **Implementation Details**:

- Promote industry-wide adoption of DNSSEC algorithms with smaller signature sizes.
- Deprecate or discourage the use of excessively large keys or algorithms where feasible.

- **Advantages**:
  - Reduces the attack surface by minimizing the impact of large RRsets.
  - Aligns with cryptographic best practices.
- **Potential Drawback**:
  - Requires long-term advocacy and gradual adoption by DNS operators.

---

6. Implement Per-Zone Resource Quotas

Resolvers could apply quotas on cache usage per zone to prevent a single zone from monopolizing cache space.

- **Implementation Details**:
  - Allocate a maximum cache size for any individual zone (e.g., 1 MB).
  - Evict or truncate entries for zones that exceed their quota.
- **Advantages**:
  - Isolates the impact of a single zone on the overall resolver cache.
  - Protects against cache exhaustion without limiting response processing globally.
- **Potential Drawback**:
  - Requires careful tuning to avoid unintended side effects on large legitimate zones.

---

7. Rate-Limit or Log Zones with Excessive DNSSEC Overhead

Resolvers could monitor and rate-limit queries for zones that frequently produce oversized or excessive DNSSEC responses.

- **Implementation Details**:

- Introduce rate-limiting or logging for zones with unusually high DNSSEC overhead.
- Automatically throttle queries for problematic zones.

- **Advantages**:
  - Helps identify and mitigate abusive patterns in real-time.
  - Adds visibility for DNS administrators to address issues proactively.
- **Potential Drawback**:
  - May require additional monitoring infrastructure.

Conclusion

The most logical and targeted mitigation for this attack is to **limit the number of signatures or total size of RRsets that resolvers process or cache**, as it directly addresses the root cause of cache exhaustion while maintaining compatibility with legitimate DNSSEC operations. Combining this with **per-zone resource quotas** and **promoting compact DNSSEC algorithms** ensures a balanced approach that safeguards the resolver's cache without disrupting legitimate DNS functionality.

> Let's say we put limit to number signatures per RRSet. Then another possible attack is :
>
> ANY Type Responses: Leveraging ANY queries to retrieve all RRSets, which generates large responses.
>
> What would be the most logical mitigation for this conceptual attack that does not affect the other parts of the DNS system?

Mitigating the abuse of **ANY** queries, which can generate large responses and amplify attacks, requires balancing functionality and security. Here's the most logical approach to mitigate this issue without significantly affecting legitimate DNS operations:

---

1. Restrict or Minimize Support for ANY Queries

Resolvers or authoritative servers can restrict the handling of **ANY** queries to prevent abuse.

Implementation Details:

- **Resolvers:**
  - Respond to **ANY** queries with an empty response or an error (e.g., `REFUSED`).
  - Allow exceptions for trusted sources or internal queries if necessary.
- **Authoritative Servers:**
  - Respond to **ANY** queries with a minimal set of records (e.g., only `A` and `AAAA`).
  - Avoid including unnecessary resource records in responses.

Advantages:

- Reduces the potential for large response amplification.
- Simple and widely implemented (e.g., some DNS implementations already do this by default, like BIND and Unbound).

Potential Drawback:

- Certain legitimate use cases for **ANY** queries (e.g., diagnostic tools) may be impacted.

---

2. Enable DNS Response Rate Limiting (RRL)

Limit the rate at which large responses (like those triggered by **ANY** queries) are sent from authoritative servers to a single client or network.

Implementation Details:

- Configure authoritative servers to apply RRL based on the client IP, query type, or response size.
- Throttle responses when a threshold (e.g., queries per second) is exceeded.

Advantages:

- Mitigates abuse while allowing legitimate queries to proceed at reasonable rates.
- Does not interfere with DNSSEC or other normal operations.

Potential Drawback:

- Adds complexity to server configuration.
- May unintentionally delay legitimate high-volume queries.

---

3. Return Truncated Responses for ANY Queries

Encourage resolvers to fall back to TCP for large **ANY** query responses, reducing amplification potential.

Implementation Details:

- Authoritative servers can respond with a truncated UDP message (TC flag set) for **ANY** queries that would result in large responses.
- Force resolvers to switch to TCP, which is less efficient for amplification attacks.

Advantages:

- Maintains compliance with DNS standards.
- Makes amplification attacks harder to execute due to the additional overhead of TCP.

Potential Drawback:

- May increase resolver load due to TCP fallback.

---

4. Implement Response Size Limiting

Limit the size of responses to **ANY** queries to prevent excessively large responses from being generated.

Implementation Details:

- Configure authoritative servers to cap the maximum response size for **ANY** queries.
- Exclude large RRsets (e.g., DNSSEC-related data) from such responses.

Advantages:

- Directly mitigates large response amplification.
- Easy to implement in modern DNS server software.

Potential Drawback:

- May result in incomplete responses for legitimate **ANY** queries.

---

5. Serve Stale or Summary Responses for ANY Queries

Instead of generating a full response, serve a pre-computed, static, or summary response for **ANY** queries.

Implementation Details:

- Authoritative servers can pre-generate small, static responses for **ANY** queries.
- Respond with summary information, such as the most common record types (e.g., `A`, `AAAA`).

Advantages:

- Prevents computational overhead and large responses.
- Reduces the risk of cache poisoning since responses are predictable.

Potential Drawback:

- Diagnostic tools relying on **ANY** queries may lose access to detailed records.

---

6. Educate Operators and Deprecate ANY Queries

Encourage the DNS community to phase out reliance on **ANY** queries for operational purposes, similar to how certain other query types have been deprecated.

Implementation Details:

- Provide best-practice guidelines for DNS operators to discourage **ANY** queries.
- Update client software to avoid generating **ANY** queries unless explicitly needed.

Advantages:

- Long-term reduction in the use of **ANY** queries.
- Aligns with modern DNS usage trends.

Potential Drawback:

- Requires time for adoption and behavior changes.

---

7. DNSSEC-Specific Adjustments

If the query involves DNSSEC-signed zones, limit the DNSSEC-related RRsets included in **ANY** query responses.

Implementation Details:

- Return only essential DNSSEC data in response to **ANY** queries.
- Avoid including large RRSIG or DNSKEY records unless explicitly queried.

Advantages:

- Prevents DNSSEC amplification attacks while maintaining zone integrity.
- Ensures compatibility with DNSSEC-compliant systems.

Potential Drawback:

- Adds complexity to authoritative server configurations.

Recommended Mitigation Strategy

The most logical approach is to **restrict or minimize support for ANY queries**, combined with **response size limiting** and **rate-limiting policies**. This ensures a balance between security and functionality:

1. **Resolvers**: Respond to **ANY** queries with minimal or empty responses, or entirely block them.
2. **Authoritative Servers**: Limit response sizes and enforce rate-limiting to avoid abuse.
3. **Fallback to TCP**: For large responses, enforce TCP fallback to mitigate amplification potential.

This strategy ensures compatibility with most legitimate use cases while protecting against abuse.

are there other solutions for this ANY type variant?

Yes, there are other solutions to address the **ANY query variant** while minimizing disruption to legitimate DNS operations. These solutions focus on balancing security, functionality, and compatibility with current DNS standards.

---

1. Use Query Substitution

Substitute **ANY** queries with a query for specific record types or respond with a single, most-relevant record type.

Implementation Details:

- **Resolvers**:
  - When receiving an **ANY** query, substitute it with a query for the most commonly requested record type (e.g., `A`, `AAAA`, or `MX`).
- **Authoritative Servers**:
  - Respond with a single relevant record type instead of the full RRset.
  - Example: Return `A` and `AAAA` records for IPv4/IPv6 compatibility.

Advantages:

- Preserves resolver functionality without exposing all records.
- Reduces the size of the response while maintaining usability.

Potential Drawback:

- May break tools that rely on the full response from **ANY** queries.

---

2. Implement DNS Cache Partitioning

Resolvers can partition their cache based on query type or client source to prevent cache pollution from **ANY** query abuse.

Implementation Details:

- Maintain separate caches for different query types, ensuring that **ANY** responses do not flush other cached entries.
- Limit the caching of **ANY** responses to avoid occupying excessive space.

Advantages:

- Prevents cache exhaustion caused by excessive **ANY** queries.
- Maintains resolver performance for other query types.

Potential Drawback:

- Increased memory usage due to partitioned caching.

---

3. Restrict ANY Queries to Authenticated Clients

Implement access control policies that allow **ANY** queries only from trusted or authenticated sources.

Implementation Details:

- Use source IP filtering or access control lists (ACLs) to allow **ANY** queries from specific trusted networks or clients.
- Block or limit responses to untrusted sources.

Advantages:

- Ensures **ANY** queries are only used for legitimate purposes.
- Reduces abuse potential without affecting trusted clients.

Potential Drawback:

- Requires additional configuration and management for access control.

---

4. Use DNS Query Minimization

Resolvers can minimize the information they request from authoritative servers to reduce the size of responses.

Implementation Details:

- When processing **ANY** queries, resolvers can fetch only specific record types from authoritative servers.
- This approach avoids returning all records in a single response.

Advantages:

- Limits the size of responses for **ANY** queries.
- Preserves the privacy and security benefits of query minimization.

Potential Drawback:

- May increase query latency due to multiple queries for specific record types.

---

5. Perform Response Fragmentation Control

Servers can carefully manage response fragmentation to reduce the risk of large, fragmented responses being abused.

Implementation Details:

- Break large responses into smaller chunks to prevent oversized packets from being abused in amplification attacks.
- Use the EDNS0 mechanism to negotiate response size limits with resolvers.

Advantages:

- Reduces amplification potential while adhering to DNS standards.
- Improves compatibility with clients and resolvers that enforce size limits.

Potential Drawback:

- Fragmented responses may require TCP fallback, increasing resolver load.

---

6. Enable Client Subnet Filtering

Resolvers can use the EDNS Client Subnet (ECS) extension to limit responses based on the client's location or subnet.

Implementation Details:

- Respond to **ANY** queries with data specific to the client’s subnet or region.
- Avoid sending unnecessary global data.

Advantages:

- Reduces the size of responses to **ANY** queries.
- Maintains compatibility with DNS infrastructure using ECS.

Potential Drawback:

- Requires ECS support and configuration on resolvers and authoritative servers.

---

7. Add Intelligent Traffic Analysis and Mitigation

Integrate DNS traffic monitoring tools to detect and mitigate abuse patterns related to **ANY** queries.

Implementation Details:

- Use DNS security appliances or intrusion detection systems to flag and throttle abusive **ANY** queries.
- Automatically apply rate limits or block abusive sources.

Advantages:

- Dynamically adapts to attack patterns in real time.
- Reduces the impact of volumetric attacks without affecting legitimate users.

Potential Drawback:

- May require additional hardware or software investments.

---

8. Utilize DNS Over HTTPS (DoH) or DNS Over TLS (DoT)

Encourage the use of encrypted DNS protocols that make it harder to abuse **ANY** queries for amplification attacks.

Implementation Details:

- Deploy DoH or DoT resolvers that restrict **ANY** queries or process them securely.
- Authoritative servers can prefer interactions over encrypted channels for additional control.

Advantages:

- Provides additional security against spoofing and interception.
- Limits abuse by requiring encrypted communication channels.

Potential Drawback:

- Not all clients and servers support DoH or DoT.

---

9. Precompute Summarized Responses

For authoritative servers, precompute summarized responses for **ANY** queries to reduce real-time computation overhead.

Implementation Details:

- Use a predefined set of summarized records for **ANY** responses.
- Return these records regardless of the actual RRsets.

Advantages:

- Reduces server load while providing a consistent response.
- Limits response sizes and simplifies processing.

Potential Drawback:

- Precomputed responses may not reflect real-time changes in DNS records.

---

10. Require EDNS0 Support for ANY Queries

Mandate that clients support EDNS0 to handle large responses gracefully, avoiding fallback issues.

Implementation Details:

- Respond with an EDNS0 extended message indicating supported features.
- Drop non-EDNS0 queries to reduce abuse potential.

Advantages:

- Ensures compatibility with modern DNS implementations.
- Reduces fragmentation and amplification risks.

Potential Drawback:

- Older clients without EDNS0 support may face compatibility issues.

Conclusion

The best solutions for mitigating the **ANY query variant** depend on the specific DNS role (resolver or authoritative server). A **multi-layered approach** combining query substitution, rate-limiting, response size control, and traffic analysis provides robust protection against abuse while preserving legitimate DNS functionality.

For resolvers, **restricting or substituting ANY queries** is the simplest and most effective solution. For authoritative servers, **response size limiting** and **rate limiting** offer practical defenses.

1. Limit the Number of Signatures Per RRset

Resolvers could impose a cap on the number of DNSKEYs or RRSIGs per RRset that are processed or cached.

Implementation Details:
Set a reasonable maximum number of DNSKEYs or RRSIGs per RRset (e.g., 10–20).
Ignore or discard additional signatures beyond the configured limit during validation or caching.
Advantages:
Directly limits the attack vector of excessive signatures.

Has minimal impact on legitimate zones that do not require an excessive number of keys.

Potential Drawback:

Might affect zones with legitimate use cases for large numbers of keys or signatures.

Can we utilize this for ANY type responses?

Yes, the concept of limiting the number of DNSKEYs or RRSIGs per RRset can be extended to mitigate abuse of **ANY type responses**, but it requires careful adaptation. Here's how this strategy can be utilized:

Adapting the Limit for ANY Type Responses

Since **ANY type queries** can aggregate multiple RRsets in a single response, the principle of limiting excessive records can be applied to reduce the response size and prevent abuse.

Implementation Details:

1. **Limit RRSIGs per RRset for ANY Queries**:
   - Apply a cap on the number of RRSIGs that are included in responses to **ANY** queries.
   - Example: If an RRset has 50 RRSIGs, only the first 10–20 are included in the response.
2. **Limit the Number of RRsets Included in ANY Responses**:
   - For **ANY** queries, return only a subset of the available RRsets (e.g., the most relevant types like `A`, `AAAA`, and `MX`).
   - Exclude less critical RRsets from the response (e.g., `TXT`, `NS`).
3. **Truncate Responses for ANY Queries**:
   - If the total response size exceeds a certain limit (e.g., 4 KB), truncate the response and set the **TC (Truncated)** flag.
   - Force the client to retry over TCP, reducing amplification potential.

Advantages for ANY Queries:

1. **Limits Amplification Potential**:
   - By capping the number of RRSIGs and RRsets, the response size is controlled, making the query less effective for amplification attacks.
2. **Preserves DNSSEC Validation**:
   - The resolver still validates the included RRSIGs, ensuring that the response complies with DNSSEC standards.
3. **Minimizes Disruption to Legitimate Traffic**:
   - Most legitimate zones do not require an excessive number of RRSIGs or RRsets, so this approach has minimal impact on standard operations.
4. **Reduces Cache Pollution**:
   - Prevents excessive or irrelevant RRSIGs from being cached by resolvers.

Potential Drawbacks:

1. **Impact on Legitimate Use Cases**:
   - Some zones with valid reasons for having a large number of RRSIGs or RRsets (e.g., large-scale multi-signature setups) may face truncation or incomplete responses.
   - Tools relying on **ANY** queries for diagnostics may not retrieve the full dataset.
2. **Resolver Complexity**:
   - Implementing and managing caps for **ANY** responses adds some complexity to resolver and server configurations.

Practical Considerations:

- **Fallback to TCP**:
  - Ensure clients can retrieve the full dataset using TCP if truncation occurs.

    - Legitimate queries are not completely blocked but require additional effort to retrieve large responses.
- **Selective Enforcement**:
    - Apply limits specifically to **ANY** queries or responses exceeding a configured size threshold.
    - Regular queries for specific record types remain unaffected.
- **Monitoring and Adaptation**:
    - Continuously monitor the impact of this limit on both legitimate and malicious traffic.
    - Adjust thresholds (e.g., number of RRSIGs, size caps) based on observed behavior.

Conclusion:

Limiting the number of DNSKEYs, RRSIGs, or RRsets included in responses to **ANY queries** is a viable mitigation strategy. It helps control response size and reduce amplification potential while maintaining compliance with DNSSEC and minimizing disruption to legitimate traffic. This strategy can be combined with other measures like rate-limiting or response truncation for a comprehensive defense.